Лабораторная работа №1

Исследование влияния предобработки данных на качество линейной регрессии

Дисциплина: Численные методы анализа

Выполнили: Колпакова Дарья Сергеевна, Недозорова Алиса Алексеевна

Группа: J3112

Вариант: 11

Санкт-Петербург, 2026

1. Цель работы

Исследовать, как предобработка признаков (масштабирование и полиномиальное расширение) влияет на:

- качество линейной регрессии по метрике `MAE`;
- устойчивость решения задачи МНК по числу обусловленности матрицы `X^T X`.

Целевая переменная по условию: `median_house_value`.

---

2. Краткие теоретические сведения

2.1 Линейная регрессия и метод наименьших квадратов

Линейная модель:

$$\hat{y} = a_0 + a_1 x_1 + \cdots + a_d x_d.$$

В матричной форме:

$$\hat{\mathbf{y}} = F\mathbf{a}.$$

Параметры определяются из условия минимума:

$$\min_{\mathbf{a}} \|F\mathbf{a} - \mathbf{y}\|_2^2.$$

Это приводит к нормальным уравнениям:

$$F^T F\mathbf{a} = F^T \mathbf{y}.$$

2.2 Масштабирование признаков

Стандартизация:

$$z = \frac{x - \mu}{\sigma}.$$

Для метода МНК качество линейной модели инвариантно относительно невырожденных линейных преобразований признаков, поэтому масштабирование обычно не меняет `MAE`, но резко улучшает численную устойчивость вычислений.

2.3 Полиномиальные признаки

При расширении признакового пространства модель остается линейной по коэффициентам, но становится нелинейной по исходным признакам:

$$\hat{y} = b_0 + \sum_{j=1}^{m} b_j \phi_j(x).$$

Рост степени полинома может уменьшить ошибку, но повышает риск переобучения и ухудшает обусловленность.

2.4 Метрики качества и устойчивости

- `MAE`:

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

- показатель переобучения:

$$\Delta = MAE_{test} - MAE_{train}$$

- число обусловленности:

$$\kappa(F^T F)$$

---

3. Ход работы

3.1 Описание датасета

Использован файл `housing.csv`.

- Размер: `20640` строк.
- Исходные столбцы: `9`.
- Целевая переменная: `median_house_value`.
- Параметры разбиения (вариант 11): `test_size=0.22`, `random_state=50`.

3.2 Листинг кода с комментариями

Ниже приведен полный рабочий листинг ключевых блоков из `lab_1.ipynb`.

Блок A. Импорт библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
```

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

np.set_printoptions(precision=6, suppress=True)
pd.set_option('display.max_columns', None)
```

Блок B. Загрузка данных и подготовка признаков

```
# Загружаем исходные данные (в этом файле разделитель – точка с запятой)
df = pd.read_csv('housing.csv', sep=';')

# Целевая переменная по условию задания
target_col = 'median_house_value'
if target_col not in df.columns:
    raise ValueError(f"В данных нет столбца {target_col}. Проверьте файл
housing.csv")

numeric_cols = [col for col in df.columns if col != target_col]

X = df[numeric_cols].copy()

y = df[target_col]

# Параметры варианта 11
test_size = 0.22
random_state = 50

print(f'Размер исходного датасета: {df.shape}')
print(f'Число признаков после подготовки: {X.shape[1]}')
print(f'test_size={test_size}, random_state={random_state}')
print('\nСтолбцы исходного датасета:')
print(list(df.columns))
print('\nПервые 10 подготовленных признаков:')
print(list(X.columns[:10]))
print('\nЧисло пропусков по столбцам:')
print(df.isna().sum())
print('\nПервые 5 строк:')
display(df.head())
```

Блок C. Функция одного эксперимента

```python
def run_experiment(with_mean: bool, with_std: bool, degree: int = 1):
    """Запускает один эксперимент и возвращает основные метрики."""

    # Разбиваем данные на обучение и тест
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

    # Копии массивов, чтобы не портить исходные данные
    X_train_proc = X_train.values
    X_test_proc = X_test.values

    # Заполняем пропуски медианой по обучающей выборке
    imputer = SimpleImputer(strategy='median')
    X_train_proc = imputer.fit_transform(X_train_proc)
    X_test_proc = imputer.transform(X_test_proc)

    # Всегда явно применяем StandardScaler в соответствии с параметрами
эксперимента
    scaler = StandardScaler(with_mean=with_mean, with_std=with_std)
    X_train_proc = scaler.fit_transform(X_train_proc)
    X_test_proc = scaler.transform(X_test_proc)

    # Добавление полиномиальных признаков
    if degree > 1:
        poly = PolynomialFeatures(degree=degree, include_bias=False)
        X_train_proc = poly.fit_transform(X_train_proc)
        X_test_proc = poly.transform(X_test_proc)

    # Сохраняем количество признаков после преобразований
    n_features = X_train_proc.shape[1]

    # Обучаем модель линейной регрессии
    model = LinearRegression()
    model.fit(X_train_proc, y_train)

    # Считаем предсказания и MAE
    y_train_pred = model.predict(X_train_proc)
    y_test_pred = model.predict(X_test_proc)

    mae_train = mean_absolute_error(y_train, y_train_pred)
    mae_test = mean_absolute_error(y_test, y_test_pred)

    # Считаем число обусловленности матрицы X^T X для обучающей выборки
    xtx = X_train_proc.T @ X_train_proc
    cond_xtx = np.linalg.cond(xtx)
```

```
    return {
        'StandardScaler mean': with_mean,
        'StandardScaler std': with_std,
        'Poly degree': degree,
        'MAE train': mae_train,
        'MAE test': mae_test,
        'Число обусл.': cond_xtx,
        'n_features_after': n_features
    }
```

Блок D. Серия из 6 экспериментов

```
# Набор экспериментов: от простого к более сложному
experiments = [
    {'mean': False, 'std': False, 'degree': 1},
    {'mean': True,  'std': False, 'degree': 1},
    {'mean': False, 'std': True,  'degree': 1},
    {'mean': True,  'std': True,  'degree': 1},
    {'mean': True,  'std': True,  'degree': 2},
    {'mean': True,  'std': True,  'degree': 3},
]

results = []
for i, cfg in enumerate(experiments, start=1):
    row = run_experiment(
        with_mean=cfg['mean'],
        with_std=cfg['std'],
        degree=cfg['degree'],
    )
    row['№'] = i
    results.append(row)

results_df = pd.DataFrame(results)
results_df = results_df[['№', 'StandardScaler mean', 'StandardScaler std',
'Poly degree', 'MAE train', 'MAE test', 'Число обусл.', 'n_features_after']]

# Округляем для красивого вывода
display(results_df.round(6))
```

Блок E. Построение графиков

```
# Строим графики зависимости MAE от номера эксперимента
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```python
# График 1: обычная шкала
axes[0].plot(results_df['№'], results_df['MAE train'], marker='o', label='MAE
train')
axes[0].plot(results_df['№'], results_df['MAE test'], marker='s', label='MAE
test')
axes[0].set_xlabel('Номер эксперимента')
axes[0].set_ylabel('MAE')
axes[0].set_title('Сравнение качества моделей (линейная шкала)')
axes[0].grid(True, alpha=0.3)
axes[0].legend()

# График 2: логарифмическая шкала по оси Y
axes[1].plot(results_df['№'], results_df['MAE train'], marker='o', label='MAE
train')
axes[1].plot(results_df['№'], results_df['MAE test'], marker='s', label='MAE
test')
axes[1].set_yscale('log')
axes[1].set_xlabel('Номер эксперимента')
axes[1].set_ylabel('MAE (log scale)')
axes[1].set_title('Сравнение качества моделей (лог-шкала)')
axes[1].grid(True, alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.show()
```

```python
# Дополнительные графики для более детального анализа результатов

# Если столбец overfit_gap еще не создан, создаем его
if 'overfit_gap' not in results_df.columns:
    results_df['overfit_gap'] = results_df['MAE test'] - results_df['MAE
train']

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1) Сравнение обусловленности (логарифмическая шкала)
axes[0].bar(results_df['№'].astype(str), results_df['Число обусл.'])
axes[0].set_yscale('log')
axes[0].set_title('Cond(X^T X) по экспериментам (log)')
axes[0].set_xlabel('Номер эксперимента')
axes[0].set_ylabel('Cond(X^T X), log scale')
axes[0].grid(True, alpha=0.3)

# 2) Разрыв между test и train (признак переобучения)
```

```
axes[1].bar(results_df['№'].astype(str), results_df['overfit_gap'])
axes[1].axhline(0, color='black', linewidth=1)
axes[1].set_title('Показатель переобучения overfit_gap')
axes[1].set_xlabel('Номер эксперимента')
axes[1].set_ylabel('MAE test - MAE train')
axes[1].grid(True, alpha=0.3)

# 3) MAE_test в зависимости от степени полинома (для StandardScaler mean=True,
std=True)
plot_df = results_df[(results_df['StandardScaler mean'] == True) &
(results_df['StandardScaler std'] == True)].copy()
plot_df = plot_df.sort_values('Poly degree')
axes[2].plot(plot_df['Poly degree'], plot_df['MAE test'], marker='o',
label='mean=True, std=True')

axes[2].set_title('MAE test vs degree')
axes[2].set_xlabel('Степень полинома')
axes[2].set_ylabel('MAE test')
axes[2].set_xticks(sorted(plot_df['Poly degree'].unique()))
axes[2].grid(True, alpha=0.3)
axes[2].legend()

plt.tight_layout()
plt.show()
```

Блок F. Расчет overfit-gap и выбор лучшей конфигурации

```
# Анализируем признаки переобучения через разрыв между train и test
analysis_df = results_df.copy()
analysis_df['overfit_gap'] = analysis_df['MAE test'] - analysis_df['MAE
train']

best_idx = analysis_df['MAE test'].idxmin()
best_row = analysis_df.loc[best_idx]

print('Лучшая конфигурация по MAE test:')
print(best_row[['№', 'StandardScaler mean', 'StandardScaler std', 'Poly
degree', 'MAE train', 'MAE test', 'overfit_gap', 'Число обусл.']])

print('\nТаблица с разницей MAE test - MAE train (признак переобучения):')
display(analysis_df[['№', 'StandardScaler mean', 'StandardScaler std', 'Poly
degree', 'MAE train', 'MAE test', 'overfit_gap']].round(6))
```

3.3 Заполненная таблица результатов экспериментов

Таблица 1 — Основные параметры и MAE

№	mean	std	degree	MAE train	MAE test
1	False	False	1	50987.48	50737.78
2	True	False	1	50987.48	50737.78
3	False	True	1	50987.48	50737.78
4	True	True	1	50987.48	50737.78
5	True	True	2	45280.13	45587.79
6	True	True	3	41640.51	42649.04

Таблица 2 — Устойчивость и размерность

№	Cond(X^TX)	n_features
1	8.54e6	8
2	2.34e7	8
3	1.74e5	8
4	1.79e2	8
5	2.41e5	44
6	2.38e9	164

---

4. Анализ результатов

4.1 Ответы на вопросы анализа

1) Как масштабирование признаков влияет на качество модели?
Для линейной модели (N1-N4) `MAE` остается одинаковым, что ожидаемо: решение МНК инвариантно относительно невырожденных линейных преобразований признаков. Главный эффект масштабирования — резкое улучшение численной устойчивости: `Cond(X^TX)` падает до минимума в N4 (`1.787565e+02`).

2) Как изменяется качество при увеличении степени полинома?
В этом эксперименте:

- степень 2 снижает `MAE test` с `50737.784665` до `45587.787152` ;
- степень 3 снижает `MAE test` еще сильнее до `42649.043249` .

3) Есть ли признаки переобучения?
Да. По `overfit_gap = MAE_test − MAE_train`:

- N1-N4: `−249.697416`;
- N5: `307.658049` (умеренный разрыв);
- N6: `1008.531337` (наибольший разрыв).

Отрицательный разрыв в N1-N4 не противоречит теории: на конкретном сплите `random_state=50` тестовая часть оказалась «удобнее» по распределению целевой переменной. При другом `random_state` знак и величина разрыва могут измениться.

Конфигурация N6 имеет минимальные `MAE train` и `MAE test` на текущем split, поэтому она лучшая по точности в данном эксперименте. Большой `overfit_gap` для N6 следует интерпретировать как повышенный риск переобучения (более высокая чувствительность к смене выборки), а не как доказательство того, что модель уже хуже по качеству.

4) Как число обусловленности связано с качеством?
Меньшее `Cond(X^TX)` означает более устойчивое решение МНК.
В наших результатах это видно явно: у N6 при лучшем `MAE test` число обусловленности максимально (`2.375572e+09`), тогда как у N5 оно существенно меньше (`2.413764e+05`), а у N4 минимально (`1.787565e+02`). Следовательно, минимум `MAE test` и численная устойчивость не всегда достигаются одновременно.

Важно: `Cond(X^TX)` не является прямым показателем переобучения, но выступает сильным косвенным сигналом нестабильности модели. В наших экспериментах при переходе от degree=2 к degree=3 число обусловленности возрастает примерно с `2.4e+05` до `2.4e+09` (рост на четыре порядка), то есть матрица нормальных уравнений становится близкой к вырожденной, а коэффициенты МНК — чувствительными к малым изменениям данных.

5) Какая комбинация параметров дает лучший результат?
По критерию задания (`min MAE test`) лучшая конфигурация — **N6**:

- `StandardScaler mean=True`,
- `StandardScaler std=True`,
- `Poly degree=3`.

Итог интерпретации: N6 — лучшая по точности на текущем split, но N5/N4 устойчивее численно; при практическом выборе модели нужно учитывать одновременно `MAE`, `overfit_gap` и `Cond(X^TX)`.

4.2 Графики зависимости MAE от параметров

Ниже оставлены ссылки на файлы графиков, которые нужно добавить в директорию отчёта.

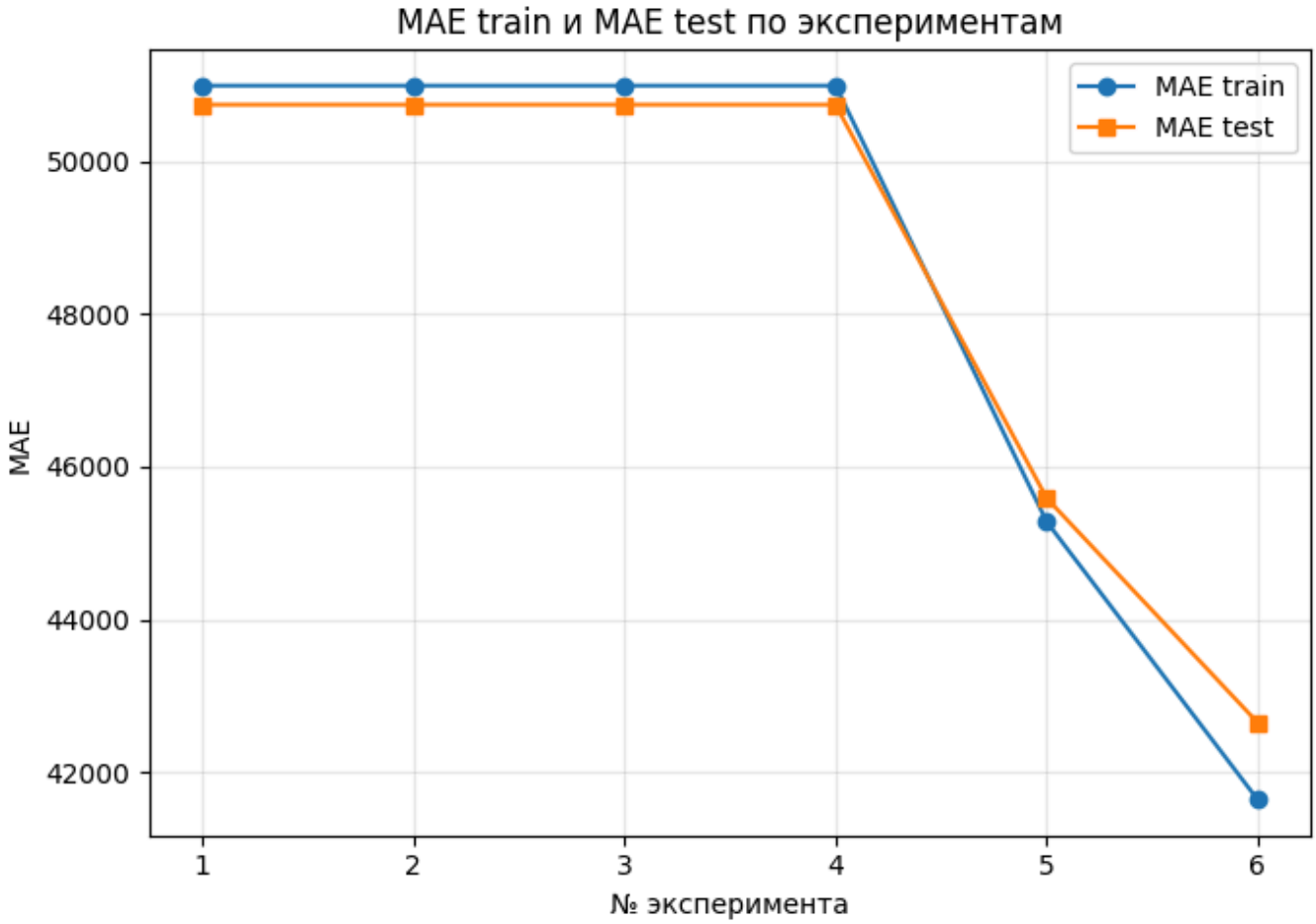


Рисунок 1 — Зависимость `MAE train` и `MAE test` от номера эксперимента.

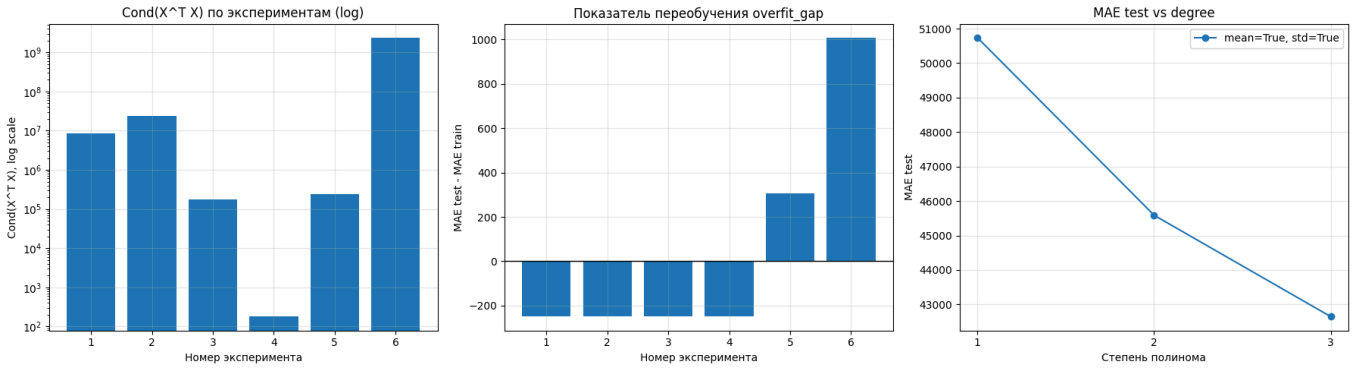


Рисунок 2 — Зависимость числа обусловленности `Cond(X^TX)` от номера эксперимента (лог. шкала). Разрыв `MAE test − MAE train` для всех 6 конфигураций. `MAE test` в зависимости от степени полинома при `mean=True`, `std=True`.

4.3 Выводы о влиянии предобработки на качество модели

- Масштабирование критично для численной устойчивости, но само по себе не улучшает MAE в линейной модели.

- Полиномиальные признаки степени 2 и 3 дают существенное улучшение качества на тесте.
- Степень 3 дает лучший `MAE test`, но сопровождается резким ростом `Cond(X^TX)` и разрыва train/test.
- На текущем split модель N6 является лучшей по точности, но N5/N4 более устойчивы численно; поэтому в оценке качества нужно учитывать не только MAE, но и `overfit_gap` с `Cond(X^TX)`.

---

5. Выводы по проделанной работе

1. Построена серия из шести согласованных экспериментов с явным применением `StandardScaler` и контролем параметров предобработки.
2. Подтверждено, что масштабирование улучшает обусловленность системы МНК на несколько порядков.
3. Полиномиальное расширение повышает точность модели: минимальный `MAE test` получен при `degree=3`.
4. Конфигурация N6 обеспечивает лучший `MAE`, но имеет максимальный `overfit_gap` и наихудшую обусловленность (`Cond(X^TX)=2.375572e+09`), что указывает на повышенный риск потери устойчивости.
5. По критерию задания итоговая лучшая конфигурация: `mean=True`, `std=True`, `degree=3` (эксперимент N6), при этом в практическом применении выбор нужно подтверждать с учетом `overfit_gap` и `Cond(X^TX)`.